

DefectIQ

AI-Powered Real-Time Industrial Quality Intelligence System

Real-Time Industrial Defect Detection & Quality Control Assistant

Frontend Application

An AI-powered industrial quality-control web application built to help manufacturing environments inspect products, detect visible defects, classify quality issues, monitor inspection results, and maintain traceable quality records.

The frontend is designed as a product-independent industrial quality platform. The current AI prototype may use a specific manufacturing dataset, but the user interface is designed around generic concepts such as Product, Batch, Inspection, Defect, Quality Status, Operator, and Production.

1. Project Overview

The frontend provides a complete web interface for the AI-powered inspection system.

The application connects the following workflow:

User Login ↓ Dashboard ↓ Product / Batch Selection ↓ Live or Image-Based Inspection ↓ AI Defect Detection ↓ Classification ↓ Severity Assessment ↓ PASS / FAIL / MANUAL REVIEW ↓ Alert ↓ Inspection Record ↓ Analytics ↓ Reports ↓ Quality Improvement

The core project workflow is:

DETECT → CLASSIFY → ASSESS → ALERT → ANALYSE → PREVENT

2. Frontend Goals

The frontend must:

- Provide a professional industrial dashboard.
- Support Admin and Operator roles.
- Allow users to start product inspections.
- Support image upload and live camera inspection.
- Display AI detection results visually.
- Show detected defects using bounding boxes.
- Display confidence and severity separately.
- Provide PASS / FAIL / MANUAL REVIEW results.
- Allow operators to remove or reject defective products.
- Automatically continue inspection when no defect is detected.

- Maintain inspection history.
- Provide batch-based quality monitoring.
- Display defect trends and analytics.
- Provide reports.
- Show AI model performance.
- Support responsive desktop, tablet, and mobile layouts.
- Keep the interface reusable for different manufacturing products.

3. Technology Stack

Category	Technology
Framework	Next.js + React
Language	TypeScript
Styling	Tailwind CSS
UI Components	shadcn/ui
Animation	Motion / Framer Motion
Icons	Lucide React
Charts	Recharts
Forms	React Hook Form
Validation	Zod
3D / Advanced Visuals	React Three Fiber / Three.js
API Communication	Fetch / Axios
State Management	React Context / Zustand if required
Authentication	Backend API / JWT
Package Manager	npm / pnpm

All selected frontend technologies should use their free/open-source availability where applicable.

Three.js should only be used when a 3D visual provides real value. The core dashboard and inspection interface should remain fast and simple.

4. Design Philosophy

The application should look like a modern industrial SaaS platform.

It should NOT look like:

- A simple college CRUD project
- A basic AI demo
- A steel-only website
- A simple image-upload website

It should look like:

"AI Industrial Quality Control Platform"

The interface should communicate:

- Intelligence
- Reliability
- Real-time monitoring
- Industrial automation
- Quality control
- Traceability
- Data-driven decision making

5. Main Design Principles

5.1 Desktop First, Fully Responsive

The primary interface is designed for a landscape desktop monitor because industrial operators and supervisors will generally use larger screens.

However, every page must also work on:

- Desktop
- Laptop
- Tablet
- Mobile

The application must use one responsive layout rather than maintaining separate desktop and mobile websites.

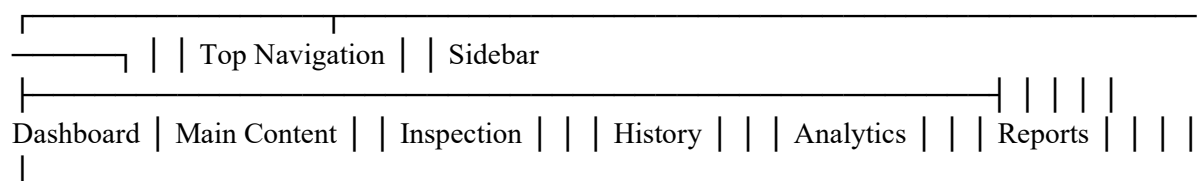
6. Responsive Layout Rules

Desktop

Use:

- Sidebar
- Top navigation
- Multi-column cards
- Large charts
- Split-screen inspection interface

Example:



Tablet

- Collapsible sidebar
- Two-column cards where possible
- Reduced chart dimensions
- Responsive tables

Mobile

- Sidebar becomes a drawer
- Top navigation remains accessible
- Cards become one-column
- Tables become cards or horizontally scrollable
- Charts resize automatically
- 60:40 inspection layout becomes vertical

7. Main Application Navigation

The application sidebar should contain:

INDUSTRIAL AI QC

MAIN |— Dashboard |— Live Inspection |— Inspections |— Batches

ANALYTICS |— Quality Analytics |— Defect Analytics |— Model Performance

TOOLS |— Image Test |— Reports

ADMIN |— Users |— Products |— Settings

The ADMIN section should only be visible to authorized users.

8. User Roles

The initial application contains two main roles:

1. Admin
2. Operator

8.1 Admin

Admin can:

- View dashboard
- Start inspections
- View inspection history
- Manage products
- Manage batches
- View analytics
- Generate reports
- View model performance
- Manage users
- Manage system settings
- Manage inspection configuration

8.2 Operator

Operator can:

- View dashboard
- Start live inspection
- Upload inspection images
- View inspection results
- Remove/reject defective products
- View inspection history
- View batches
- View quality analytics
- Generate permitted reports
- Give feedback on AI detections

Operator should not be allowed to:

- Manage users
- Change system-level permissions
- Modify critical AI configuration
- Change administrative settings

9. Authentication Pages

9.1 Login Page

Route:

/login

Purpose

Authenticate users before entering the application.

UI

Desktop:

				Industrial AI QC	Login				AI-powered quality	Email	
inspection platform	Password			Remember me					[Login]		
Password?				Create Account							Forgot

Features

- Email
- Password
- Show/hide password
- Remember me
- Login
- Forgot password
- Register link
- Validation messages
- Loading state
- Authentication error

10. Register Page

Route:

/register

Fields

- Full Name
- Email
- Password
- Confirm Password
- Role

Role Selection

Use radio buttons:

- ☐ Admin
- ☐ Operator

For the hackathon prototype this demonstrates role-based registration.

For a production system, Admin registration should normally require authorization or invitation.

Validation

Use:

React Hook Form + Zod

Validate:

- Required fields
- Valid email
- Password length
- Password confirmation
- Role selection

11. Dashboard

Route:

/dashboard

The dashboard is the main overview page.

The dashboard should answer:

"What is happening with product quality today?"

12. Dashboard KPI Cards

The top of the dashboard should contain four important cards.

Card 1

TODAY'S INSPECTED PRODUCTS

Example:

1,248

Card 2

PASSED

1,086

87.0%

Card 3

REJECTED

162

13.0%

Card 4

DEFECT RATIO

13.0%

Each card should support:

- Icon
- Main number
- Label
- Comparison/percentage if available
- Small trend indicator

13. Dashboard Charts

Inspection Trend

Line or area chart.

Shows:

- Inspected products
- Passed products
- Rejected products

over time.

Quality Distribution

Donut chart:

- Passed
- Rejected
- Manual Review

Defect Distribution

Bar chart:

Defect Type | Count

Recent Alerts

Show:

- Critical defect
- High defect rate
- Manual review required
- Inspection error

14. Recent Inspection Table

Columns:

- Product ID
- Batch ID
- Inspection Time
- Result

- Defect
- Severity
- Confidence
- Operator
- Action

Action:

[View]

Clicking View opens:

/inspections/[id]

15. Live Inspection Page

Route:

/inspection/live

This is the most important operational page.

The design must use the requested:

60 : 40 layout

LEFT = 60%

RIGHT = 40%

16. Live Inspection Desktop Layout

Live Inspection															
Product Details				LIVE CAMERA				Product ID							
DETECTION AREA				Batch ID		Status									
								AI Result				Defect		PRODUCT	
Confidence		CAMERA		Severity								[DEFECT BOX]		Defect	
Details														[Remove Product]	
Detection Status		[View Details]				[Resume]									

LEFT: 60%

RIGHT: 40%

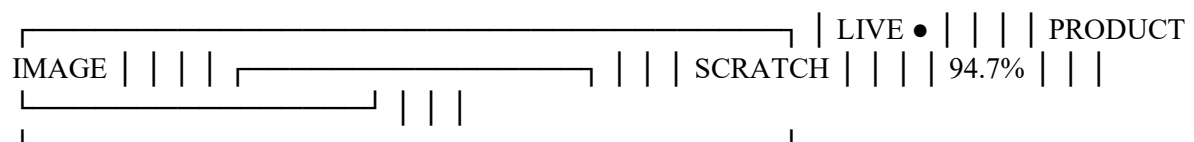
17. Left Detection Panel

The left panel is responsible for visual AI inspection.

It contains:

- Live camera feed
- Uploaded image/video preview
- Detection overlay
- Bounding boxes
- Defect labels
- Confidence values
- Detection status
- FPS if available
- Current inspection counter

Example:



Detection Status:

- Detection Active

18. Right Product Information Panel

The right panel displays information about the currently inspected product.

Product Information

- Product ID

- Product category
- Batch ID
- Inspection ID
- Inspection time
- Operator

AI Result

- Result
- Defect type
- Confidence
- Severity
- Defect count

Defect Location

- Location
- Bounding-box coordinates if required

Recommendation

Display:

- Possible cause
- Recommended process check

Important:

AI-generated possible causes must be presented as possible explanations, not confirmed root causes.

19. No-Defect Workflow

If no defect is detected:

Display:

	✓ PRODUCT PASSED				No
visible defect detected				Continuing inspection...	

The inspection should automatically continue.

The operator should NOT have to manually click "Next Product" after every successful inspection.

Workflow:

Product ↓ AI Detection ↓ No defect ↓ PASS ↓ Save result ↓ Continue Detection

20. Defect Detection Workflow

When a defect is detected:

Product ↓ AI Detection ↓ Defect Found ↓ Display Bounding Box ↓ Show Defect Information ↓ Calculate/Display Severity ↓ Raise Alert ↓ Operator Decision ↓ Remove / Review / Continue

21. Remove Defective Product

When a defect is detected, show:

Remove Product

Button:

[Remove Product]

Clicking it opens a confirmation modal.

Example:

Product: PRD-1024 | Defect: Scratch | Severity: High | This product will be marked as rejected. | [Cancel] [Confirm Removal]

Remove Product? | | |

After confirmation:

- Product becomes REJECTED
- Inspection result is saved
- Rejection reason is stored
- Alert is acknowledged
- Detection can resume

22. Continue Detection

After a defective product is removed:

[Resume Detection]

The system returns to:

- Detection Active

and starts inspecting the next product.

This creates a continuous inspection loop.

23. Mobile Live Inspection

On desktop:

60% camera 40% details

On mobile:

100% camera ↓ 100% details

Example:

				Live Inspection										
						CAMERA				100%				
				Product Information							Result		Defect	
Severity			Confidence				[Remove Product]							

The 60:40 ratio is therefore preserved conceptually while the layout changes responsively.

24. Inspection Result Detail Page

Route:

/inspections/[id]

This page shows the complete record of one inspection.

Left Section

Inspection image.

Show:

- Original image
- Detection overlay
- Bounding boxes
- Defect labels
- Confidence

Right Section

Inspection information:

- Inspection ID
- Product ID
- Batch ID
- Date
- Time
- Result
- Defect type
- Severity
- Confidence
- Operator
- Model version

25. Inspector Verification

The result page should support:

AI Result:

"Defect Detected"

Then:

Was this result correct?

[✓ Correct]

[✕ False Alarm]

If corrected:

Allow:

- Final classification
- Remarks
- Reviewer
- Verification time

The original AI result must remain stored.

26. Inspection Statuses

Use four statuses:

PASS

No relevant visible defect detected.

FAIL

A defect violates configured quality rules.

MANUAL REVIEW

AI result is uncertain or requires human verification.

INSPECTION ERROR

Inspection could not be completed.

Important:

Inspection Error must NEVER be treated as PASS.

27. Inspection History Page

Route:

/inspections

Purpose:

Search and review all previous inspections.

Filters

- Date
- Product
- Batch
- Result
- Defect
- Severity
- Operator

Search

Search by:

- Product ID
- Inspection ID
- Batch ID

Table

Product ID Batch Time Result Defect Severity Confidence Operator Action

Action:

[View]

28. Product Management Page

Route:

/products

Purpose:

Manage generic manufacturing products.

Features:

- Product list

- Add product
- Edit product
- Product category
- Quality requirements
- Product inspection history

Product fields:

- Product ID
- Product name
- Category
- Description
- Quality rules
- Active/inactive

The UI should use "Product", not "Steel Product".

29. Batch Management

Route:

/batches

Features:

- Create batch
- Select batch
- Batch status
- Production date
- Shift
- Product quantity
- Inspected quantity
- Passed quantity
- Rejected quantity
- Defect ratio

Example:

BATCH-102

Total:

1,240

Passed:

1,102

Rejected:

138

Defect Ratio:

11.1%

30. Batch Detail Page

Route:

/batches/[id]

Display:

- Batch information
- Product information
- Inspection progress
- Pass/fail ratio
- Defect distribution
- Inspection timeline
- Repeated defects
- Batch quality score

Actions:

[View Inspections]

[Generate Report]

31. Quality Analytics Page

Route:

/analytics

Purpose:

Provide overall quality insights.

Charts:

- Inspection volume
- Pass rate
- Reject rate
- Defect rate
- Quality trend
- Batch comparison
- Shift comparison

Use Recharts.

Charts must be:

- Responsive
- Interactive
- Tooltip enabled
- Legend enabled
- Accessible

32. Defect Analytics Page

Route:

/analytics/defects

Show:

Defect Distribution

Bar chart.

Defects Over Time

Line chart.

Severity Distribution

Donut chart.

Defects by Batch

Bar chart.

Repeated Defect Pattern

Highlight defects appearing repeatedly.

If a defect increases suddenly:

Display:

⚠ Quality Risk

"Defect frequency has increased above the configured threshold."

33. Alerts Page

Route:

/alerts

Features:

- Critical defect alerts
- High severity alerts
- Repeated defect alerts
- Inspection errors
- Manual review alerts

Each alert contains:

- Alert type
- Product
- Batch
- Time
- Severity
- Status
- Acknowledge action

34. Notification Center

A notification dropdown should also be available globally.

Top-right:



Example:

CRITICAL DEFECT Product PRD-1024

HIGH DEFECT RATE Batch BATCH-102

MANUAL REVIEW REQUIRED Inspection #INSP-2034

35. Manual Review Page

Route:

/reviews

This page is for uncertain AI results.

Show:

- Inspection image
- AI prediction
- Confidence
- Suggested defect
- Severity
- Reviewer decision

Actions:

[Confirm]

[Correct]

[Reject Prediction]

[Mark for Further Review]

36. Reports Page

Route:

/reports

Reports:

- Daily quality report
- Weekly quality report
- Monthly quality report
- Product report
- Batch report
- Defect report

Actions:

[Generate PDF]

[Export Excel]

37. Report Preview

Before downloading:

Show report preview containing:

- Report title
- Reporting period
- Total inspected
- Passed
- Rejected
- Defect ratio
- Defect distribution
- Quality trends
- Critical alerts

38. Model Performance Page

Route:

/model-performance

This page is mainly for Admin.

Show:

- Model name
- Model version
- Training information
- Precision
- Recall
- F1 Score
- mAP if available
- Confusion matrix
- Class-wise performance

Example:

Class	Precision	Recall
Class A	94%	91%
Class B	92%	89%
Class C	95%	93%

The actual values must come from the backend/model evaluation. Do not hard-code production performance values.

39. Image Test Page

Route:

/test

Purpose:

Allow offline AI testing.

Options:

[Upload Image]

[Upload Video]

After upload:

Uploading ↓ Processing ↓ AI Detection ↓ Result

Display:

- Image
- Bounding boxes
- Defect
- Confidence
- Severity
- Result

40. User Management Page

Route:

/admin/users

Admin only.

Features:

- User list
- Add user
- Edit user
- Activate/deactivate
- Assign role
- Search
- Filter

Columns:

Name Email Role Status Created Last Login Action

41. Settings Page

Route:

/settings

Sections:

Profile

- Name
- Email
- Role

Inspection Settings

- Confidence threshold
- Severity rules
- Alert threshold

Camera

- Camera source
- Resolution
- FPS

Notifications

- Sound
- Browser
- Email

Security

- Password
- Session settings

42. Audit Log

Route:

/admin/audit

Admin only.

Track:

- Login
- Logout

- Inspection decision
- Product removal
- Result correction
- User changes
- Settings changes

Each entry:

User Action Date Time Record Reason

43. Help Page

Route:

/help

Include:

- How to start inspection
- How to remove a defective product
- How to interpret confidence
- How severity works
- How to review inspection history
- How to generate reports

44. Important AI UI Rules

The frontend must clearly distinguish:

AI Confidence

How confident the model is in its prediction.

Example:

Confidence: 94.7%

Defect Severity

How serious the defect is according to quality rules.

Example:

Severity: HIGH

These two values must never be treated as the same thing.

45. Multiple Defects

One product may contain multiple defects.

Therefore the UI must support:

Defect 1

- Scratch
- High
- 94%

Defect 2

- Surface issue
- Medium
- 87%

The image should display multiple bounding boxes.

The right-side panel can display:

Detected Defects: 2

and list each defect.

46. Inspection Error Handling

If the AI service fails:

DO NOT display:

PASS

Instead:

| ⚠ INSPECTION ERROR | | | |

Inspection could not be | | completed. | | | | [Retry] [Manual Review] |

47. Poor Image Handling

If image quality is insufficient:

Display:

⚠ Image Quality Warning

"The image is unclear for reliable inspection."

Actions:

[Capture Again]

[Upload Another Image]

[Manual Review]

Never automatically mark it as PASS.

48. Frontend API Architecture

Frontend communicates with the backend through APIs.

Example:

/api/auth/login

/api/auth/register

/api/products

/api/batches

/api/inspections

/api/inspections/{id}

/api/detection

/api/alerts

/api/analytics

/api/reports

/api/model-performance

/api/users

49. Real-Time Detection Communication

For live inspection, normal REST requests may not be sufficient for continuous updates.

Use WebSocket or another real-time communication mechanism when the backend supports it.

Example:

Camera ↓ Backend ↓ AI Model ↓ WebSocket ↓ Frontend ↓ Bounding Box + Result

This allows the frontend to update the detection screen without continually refreshing the page.

50. Suggested Next.js Structure

frontend/

```
├── app/ | | ├── page.tsx | | | ├── login/ | | | ├── page.tsx | | | ├── register/ | | | └──
page.tsx | | | ├── dashboard/ | | | ├── page.tsx | | | ├── inspection/ | | | ├── live/ | | |
├── page.tsx | | | ├── [id]/ | | | ├── page.tsx | | | ├── inspections/ | | | ├── page.tsx | | |
├── products/ | | | ├── page.tsx | | | ├── [id]/ | | | ├── page.tsx | | | ├── batches/ | | |
├── page.tsx | | | ├── [id]/ | | | ├── page.tsx | | | ├── analytics/ | | | ├── page.tsx | | |
├── defects/ | | | ├── page.tsx | | | ├── alerts/ | | | ├── page.tsx | | | ├── reviews/ | | |
├── page.tsx | | | ├── reports/ | | | ├── page.tsx | | | ├── test/ | | | ├── page.tsx | | |
├── model-performance/ | | | ├── page.tsx | | | ├── settings/ | | | ├── page.tsx | | |
admin/ | | ├── users/ | | | ├── page.tsx | | | ├── audit/ | | | ├── page.tsx | | | ├── components/ |
├── layout/ | | ├── navigation/ | | ├── dashboard/ | | ├── inspection/ | | ├── products/ | |
batches/ | | ├── analytics/ | | ├── alerts/ | | ├── reports/ | | ├── ui/ | | ├── lib/ | | ├── api.ts |
├── auth.ts | | ├── utils.ts | | ├── constants.ts | | ├── hooks/ | | ├── useAuth.ts | |
useInspection.ts | | ├── useDetection.ts | | ├── useWebSocket.ts | | ├── types/ | | ├── user.ts |
├── product.ts | | ├── batch.ts | | ├── inspection.ts | | ├── defect.ts | | ├── schemas/ | |
auth.ts | | ├── product.ts | | ├── inspection.ts | | ├── public/ | | ├── images/ | | ├── icons/ |
```

|— styles/ | |— globals.css | |— package.json |— tsconfig.json |— next.config.ts |—
README.md

51. Component Structure

Reusable components should be created instead of repeating UI.

Examples:

Dashboard:

- KPI Card
- Trend Chart
- Defect Chart
- Recent Inspection Table
- Alert Card

Inspection:

- Camera Viewer
- Detection Overlay
- Detection Status
- Product Information
- Defect Card
- Severity Badge
- Confidence Badge
- Remove Product Modal
- Inspection Controls

Global:

- Sidebar
- Header
- User Menu
- Notification Center
- Loading State
- Error State
- Confirmation Modal

52. UI Components

Use shadcn/ui for:

- Button
- Card
- Badge
- Dialog
- Dropdown Menu
- Input
- Select
- Radio Group
- Checkbox
- Tabs
- Table
- Tooltip
- Alert
- Progress
- Sheet
- Skeleton

Customize them using Tailwind CSS to match the industrial dashboard design.

53. Color System

Use a restrained professional industrial palette.

Suggested semantic colors:

Primary:

- Brand color

Success:

- PASS

Warning:

- MANUAL REVIEW

Danger:

- DEFECT / REJECTED

Neutral:

- Processing
- Information

Do not use excessive colors.

Color should communicate status rather than decorate the interface.

54. Typography

Recommended:

- Modern sans-serif font
- Clear hierarchy
- Large dashboard numbers
- Strong page headings
- Compact table text

Example:

Page title: 32px

Section heading: 20–24px

Body: 14–16px

Dashboard KPI: 28–36px

Mobile typography should scale responsively.

55. Animation

Use Motion / Framer Motion only where useful.

Recommended:

- Page transitions
- Card entrance
- Alert appearance
- Detection status change
- Modal transitions
- Sidebar animation
- Loading states

Do NOT animate every element.

The inspection screen should prioritize real-time performance.

56. Three.js

Three.js / React Three Fiber is optional.

Use it only for:

- Landing-page industrial visualization
- Factory/production-line visual
- 3D hero animation

Do not use Three.js for:

- Dashboard charts
- Tables
- Inspection image detection
- Normal application UI

The main application should remain lightweight.

57. Accessibility

The frontend should support:

- Keyboard navigation
- Proper button labels
- Accessible forms
- Visible focus states
- Sufficient text contrast
- ARIA labels where required
- Screen-reader-friendly status messages

58. Loading States

Every API-dependent page should have a loading state.

Example:

Dashboard:

[Skeleton KPI] [Skeleton Chart] [Skeleton Table]

Inspection:

"Connecting to inspection service..."

Detection:

"Processing frame..."

59. Empty States

Example:

No inspections:

"No inspection records found."

Button:

[Start New Inspection]

No alerts:

"You're all caught up."

No batches:

"No production batches available."

60. Error States

Errors should be understandable.

Bad:

"500 Internal Server Error"

Better:

"Inspection service is currently unavailable."

Actions:

[Retry]

[Manual Review]

61. Security

Frontend must respect backend authorization.

Never rely only on hiding UI elements.

The backend must verify:

- User identity
- User role
- Permission
- Requested resource

Frontend should use protected routes for:

- Dashboard
- Inspection
- Analytics
- Reports
- Admin

62. Core Data Models

Frontend types should represent:

User

- id
- name
- email
- role
- status

Product

- id
- name
- category

- qualityRules

Batch

- id
- productId
- productionDate
- shift
- quantity

Inspection

- id
- productId
- batchId
- result
- timestamp
- operatorId

Defect

- id
- type
- severity
- confidence
- location

AI Result

- modelVersion
- prediction
- confidence

Verification

- reviewer
- decision
- remarks

Alert

- type
- priority
- status

63. Main User Journey

Operator Journey

Login ↓ Dashboard ↓ Select Product ↓ Select Batch ↓ Live Inspection ↓ AI Detection ↓ No Defect ↓ PASS ↓ Continue Detection

OR

AI Detection ↓ Defect Found ↓ Alert ↓ View Details ↓ Remove Product ↓ REJECTED ↓ Resume Detection

64. Admin Journey

Login ↓ Dashboard ↓ Quality Analytics ↓ Inspection History ↓ Model Performance ↓ User Management ↓ Reports ↓ System Settings

65. Complete Page List

#	Page	Access	View Type
1	Landing	Public	Marketing
2	Login	Public	Authentication
3	Register	Public	Authentication
4	Dashboard	Admin/Operator	Analytics
5	Live Inspection	Admin/Operator	Real-time
6	Inspection History	Admin/Operator	Table
7	Inspection Detail	Admin/Operator	Detail
8	Product Management	Admin	CRUD
9	Product Detail	Admin/Operator	Detail

10	Batch Management	Admin/Operator	Table/Cards
11	Batch Detail	Admin/Operator	Analytics
12	Quality Analytics	Admin/Operator	Charts
13	Defect Analytics	Admin/Operator	Charts
14	Alerts	Admin/Operator	List
15	Manual Review	Admin/Operator	Review
16	Reports	Admin/Operator	Reports
17	Image Test	Admin/Operator	AI Test
18	Model Performance	Admin	AI Analytics
19	Users	Admin	Management
20	Audit Log	Admin	Table
21	Settings	Admin/Operator	Settings
22	Help	Admin/Operator	Information

66. MVP Frontend

For the hackathon, build these first:

1. Login
2. Register
3. Dashboard
4. Product Selection
5. Live Inspection
6. Inspection Result
7. Inspection History
8. Batch Management
9. Basic Analytics
10. Reports

The most important pages are:

Dashboard + Live Inspection + Inspection Detail

67. Development Priority

Phase 1

Setup:

- Next.js
- TypeScript
- Tailwind
- shadcn/ui
- ESLint
- Project structure

Phase 2

Authentication:

- Login
- Register
- Role handling
- Protected routes

Phase 3

Application layout:

- Sidebar
- Header
- Responsive navigation
- User menu

Phase 4

Dashboard:

- KPI cards
- Charts
- Recent inspections
- Alerts

Phase 5

Inspection:

- Product selection
- Batch selection
- Camera/upload
- Detection display
- 60:40 layout
- PASS/FAIL
- Remove Product
- Resume Detection

Phase 6

History:

- Inspection table
- Filters
- Detail page
- Verification

Phase 7

Analytics:

- Defect charts
- Quality trends
- Batch analytics

Phase 8

Reports:

- Report generation UI
- PDF/Excel actions

Phase 9

Admin:

- Users
- Settings
- Model performance

- Audit log

Phase 10

Polish:

- Responsive design
- Animations
- Loading states
- Error handling
- Accessibility
- Performance

68. Important Product-Independent Naming

Use:

Product Product ID Product Category Inspection Inspection ID Batch Batch ID Defect Defect Type
Quality Status Operator Admin Production Quality

Avoid making the frontend UI depend on:

Steel Steel Sheet Steel Strip Steel Product

The current dataset/model can remain specific internally.

This makes the application easier to extend to other manufacturing categories later.

69. Current AI Dataset Scope

The frontend should remain generic, but the project documentation must remain technically honest.

The current supplied project document identifies the selected dataset as a hot-rolled steel surface defect dataset and lists classes such as inclusion, patches, crazing, rolled-in scale, scratches, and pitted surface.

Therefore:

Frontend: Generic industrial product terminology.

Current AI model: Dataset-specific defect classes.

Future: Additional products and defect categories can be added.

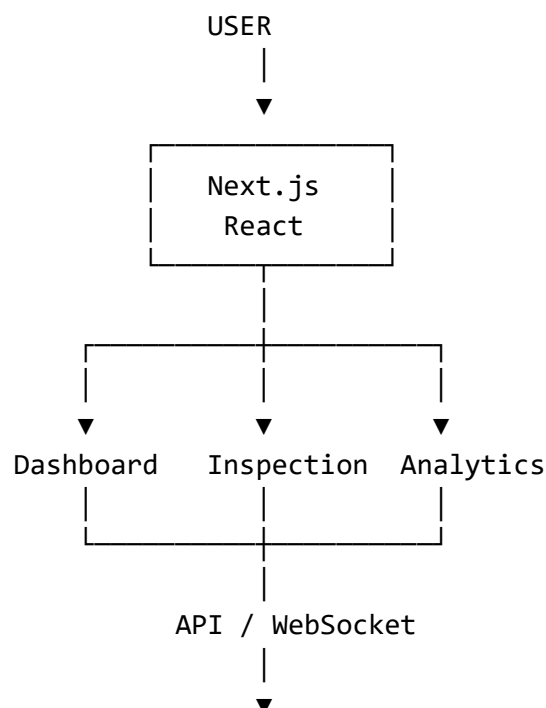
70. Important Quality Rules

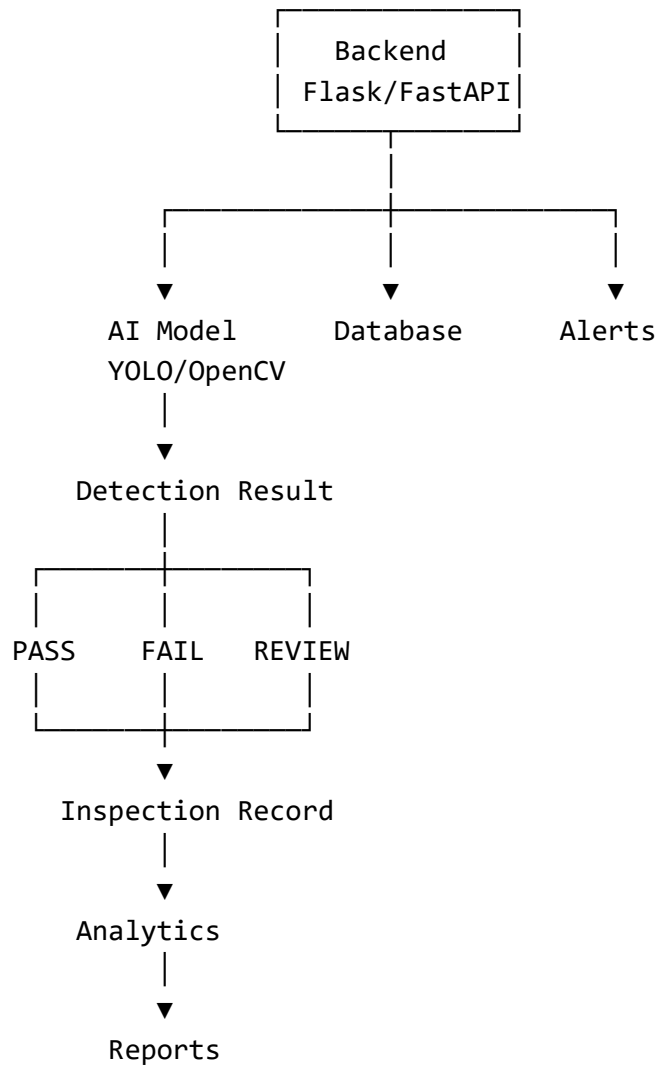
The UI must support:

- Multiple defects per product.
- AI confidence.
- Defect severity.
- Manual review.
- False alarm feedback.
- Inspection errors.
- Product rejection.
- Inspection traceability.
- Repeated defect detection.

AI confidence and defect severity must be displayed separately.

71. Final Frontend Architecture





72. Final Product Vision

The frontend should not simply show:

"AI detected a defect."

It should show the complete quality-control process:

INSPECT ↓ DETECT ↓ CLASSIFY ↓ ASSESS ↓ DECIDE ↓ ALERT ↓ REMOVE / REVIEW ↓
RECORD ↓ ANALYSE ↓ IMPROVE

The result should feel like a complete industrial quality-control platform with AI at its center.

73. Final Success Criteria

The frontend is considered complete when:

☐ User can register ☐ User can login ☐ Admin and Operator roles work ☐ Dashboard displays today's KPIs ☐ Product can be selected ☐ Batch can be selected ☐ Live inspection screen works ☐ 60:40 desktop inspection layout works ☐ Mobile inspection layout works ☐ AI detection result is displayed ☐ Bounding boxes are displayed ☐ Confidence is displayed ☐ Severity is displayed ☐ PASS is displayed ☐ FAIL is displayed ☐ Manual Review is supported ☐ Detection continues after PASS ☐ Defective product can be removed ☐ Detection can resume after removal ☐ Inspection history works ☐ Product details work ☐ Batch details work ☐ Analytics work ☐ Alerts work ☐ Reports work ☐ Image testing works ☐ Model performance page works ☐ Admin user management works ☐ Settings work ☐ Loading states exist ☐ Error states exist ☐ Empty states exist ☐ Responsive design works ☐ Role-based access works ☐ API integration works ☐ UI is accessible ☐ Application is production-quality in appearance

And use color for front end like : Steel-blue & Blue-violet) or Peach, orange & white)

means you use colors but theme is light not dark but also a color that highlight our project.